

Markov Decision Processes and Dynamic Programming

Lecture Notes

1 Introduction and Recap

Reinforcement learning is the science of learning to make decisions. The fundamental components of an RL agent include:

- A **policy**: mapping from states to actions
- A **value function**: expected cumulative reward
- A **model**: agent's representation of the environment

The general problem involves taking into account time and consequences. Decisions affect the reward, the agent state, and the environment state.

1.1 Scope of This Lecture

- Previous lecture: multiple actions, but only one state (no model)
- This lecture:
 - Formalize the problem with full sequential structure
 - Discuss dynamic programming methods assuming the true model is given
- Future lectures: use similar ideas with sampling instead of the true model

2 Formalising the RL Interaction

We develop a mathematical formulation of the agent-environment interaction called a *Markov Decision Process* (MDP). This enables precise discussion of objectives and solution methods.

2.1 Simplifying Assumption

For now, we assume the environment is **fully observable**: the current observation contains all relevant information.

Remark 2.1. Almost all RL problems can be formalized as MDPs:

- Optimal control primarily deals with continuous MDPs
- Partially observable problems can be converted into MDPs
- Bandits are MDPs with one state

2.2 Markov Decision Process

Definition 2.2 (Markov Decision Process - Sutton & Barto Formulation). A **Markov Decision Process** is a tuple $(\mathcal{S}, \mathcal{A}, p, \gamma)$, where:

- $\mathcal{S}$ is the set of all possible states
- $\mathcal{A}$ is the set of all possible actions
- $p(r, s' \mid s, a)$ is the joint probability of reward r and next state s' , given state s and action a
- $\gamma \in [0, 1]$ is a discount factor that trades off later rewards to earlier ones

Remark 2.3. The function p defines the dynamics of the problem. Sometimes it is useful to marginalize:

$$p(s' \mid s, a) = \sum_r p(s', r \mid s, a) \quad (1)$$

$$\mathbb{E}[R \mid s, a] = \sum_r r \sum_{s'} p(r, s' \mid s, a) \quad (2)$$

Definition 2.4 (Markov Decision Process - Alternative Formulation). A **Markov Decision Process** is a tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$, where:

- $\mathcal{S}$ is the set of all possible states
- $\mathcal{A}$ is the set of all possible actions
- $p(s' \mid s, a)$ is the probability of transitioning to s' , given state s and action a
- $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the expected reward: $r(s, a) = \mathbb{E}[R \mid s, a]$
- $\gamma \in [0, 1]$ is a discount factor

Remark 2.5. These two formulations are equivalent; no additional assumptions are required.

2.3 The Markov Property

Definition 2.6 (Markov Property). Consider a sequence of random variables $\{S_t\}_{t \in \mathbb{N}}$ indexed by time. A state s has the **Markov property** when for all states $s' \in \mathcal{S}$:

$$p(S_{t+1} = s' \mid S_t = s) = p(S_{t+1} = s' \mid h_{t-1}, S_t = s) \quad (3)$$

for all possible histories $h_{t-1} = \{S_1, \dots, S_{t-1}, A_1, \dots, A_{t-1}, R_1, \dots, R_{t-1}\}$.

Proposition 2.7. In a Markov Decision Process, all states are assumed to have the Markov property. This means:

1. The state captures all relevant information from the history
2. Once the state is known, the history may be discarded
3. The state is a sufficient statistic of the past

Exercise 2.8 (Markov Property Implications). In an MDP, which of the following statements are true?

$$p(S_{t+1} = s' \mid S_t = s, A_t = a) = p(S_{t+1} = s' \mid S_1, \dots, S_{t-1}, A_1, \dots, A_t, S_t = s) \quad (1)$$

$$p(S_{t+1} = s' \mid S_t = s, A_t = a) = p(S_{t+1} = s' \mid S_1, \dots, S_{t-1}, S_t = s, A_t = a) \quad (2)$$

$$p(S_{t+1} = s' \mid S_t = s, A_t = a) = p(S_{t+1} = s' \mid S_1, \dots, S_{t-1}, S_t = s) \quad (3)$$

$$p(R_{t+1} = r, S_{t+1} = s' \mid S_t = s) = p(R_{t+1} = r, S_{t+1} = s' \mid S_1, \dots, S_{t-1}, S_t = s) \quad (4)$$

Example 2.9 (Cleaning Robot MDP). Consider a robot that collects soda cans:

- States: $\mathcal{S} = \{\text{high}, \text{low}\}$ (battery charge levels)
- Actions: $\{\text{wait}, \text{search}\}$ in state high; $\{\text{wait}, \text{search}, \text{recharge}\}$ in state low
- Stochastic dynamics:

$$p(S_{t+1} = \text{high} \mid S_t = \text{high}, A_t = \text{search}) = \alpha \quad (4)$$

$$p(S_{t+1} = \text{low} \mid S_t = \text{high}, A_t = \text{search}) = 1 - \alpha \quad (5)$$

- Reward: expected or actual number of collected cans

3 Formalising the Objective

3.1 Returns

Acting in an MDP results in immediate rewards R_t , which lead to returns G_t :

Definition 3.1 (Undiscounted Return). For episodic/finite horizon problems:

$$G_t = R_{t+1} + R_{t+2} + \dots + R_T = \sum_{k=0}^{T-t-1} R_{t+k+1} \quad (6)$$

Definition 3.2 (Discounted Return). For finite or infinite horizon problems:

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t} R_T = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1} \quad (7)$$

Definition 3.3 (Average Return). For continuing, infinite horizon problems:

$$G_t = \frac{1}{T-t-1} (R_{t+1} + R_{t+2} + \dots + R_T) = \frac{1}{T-t-1} \sum_{k=0}^{T-t-1} R_{t+k+1} \quad (8)$$

Remark 3.4. Returns are random variables that depend on the MDP and the policy.

3.2 Discounted Return

Definition 3.5 (Infinite Horizon Discounted Return). For $T \rightarrow \infty$:

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (9)$$

Proposition 3.6 (Properties of Discount Factor). The discount $\gamma \in [0, 1]$ represents the present value of future rewards:

- The marginal value of receiving reward R after $k + 1$ time-steps is $\gamma^k R$
- For $\gamma < 1$, immediate rewards are more important than delayed rewards
- γ close to 0 leads to “myopic” evaluation
- γ close to 1 leads to “far-sighted” evaluation

Remark 3.7 (Reasons for Discounting). 1. **Problem specification:**

- Immediate rewards may be more valuable (e.g., earning interest)
- Animal/human behavior shows preference for immediate reward

2. **Solution considerations:**

- Mathematically convenient
- Avoids infinite returns in cyclic Markov processes

3. Reward and discount together determine the goal

3.3 Policies

Definition 3.8 (Policy). A **policy** is a mapping $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ that, for every state s , assigns for each action $a \in \mathcal{A}$ the probability of taking that action in state s , denoted $\pi(a | s)$.

Definition 3.9 (Deterministic Policy). For deterministic policies, we write $a_t = \pi(s_t)$ to denote the action taken by the policy.

Definition 3.10 (Goal of an RL Agent). To find a behavior policy that maximizes the expected return G_t .

3.4 Value Functions

Definition 3.11 (State-Value Function). The **state-value function** $v_\pi(s)$ gives the expected return starting from state s and following policy π :

$$v_\pi(s) = \mathbb{E}[G_t | S_t = s, \pi] \quad (10)$$

Definition 3.12 (Action-Value Function). The **action-value function** $q_\pi(s, a)$ gives the expected return starting from state s , taking action a , and then following policy π :

$$q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a, \pi] \quad (11)$$

Proposition 3.13 (Relationship Between Value Functions). For all states s :

$$v_\pi(s) = \sum_a \pi(a | s) q_\pi(s, a) = \mathbb{E}[q_\pi(S_t, A_t) | S_t = s, \pi] \quad (12)$$

3.5 Optimal Value Functions

Definition 3.14 (Optimal State-Value Function). The **optimal state-value function** $v^*(s)$ is the maximum value function over all policies:

$$v^*(s) = \max_{\pi} v_{\pi}(s) \quad (13)$$

Definition 3.15 (Optimal Action-Value Function). The **optimal action-value function** $q^*(s, a)$ is the maximum action-value function over all policies:

$$q^*(s, a) = \max_{\pi} q_{\pi}(s, a) \quad (14)$$

Remark 3.16. The optimal value function specifies the best possible performance in the MDP. An MDP is “solved” when we know the optimal value function.

3.6 Optimal Policies

Definition 3.17 (Partial Ordering on Policies). Define a partial ordering over policies:

$$\pi \geq \pi' \iff v_{\pi}(s) \geq v_{\pi'}(s), \quad \forall s \in \mathcal{S} \quad (15)$$

Theorem 3.18 (Existence of Optimal Policies). *For any Markov decision process:*

1. *There exists an optimal policy π^* such that $\pi^* \geq \pi$ for all policies π . (There may be more than one optimal policy.)*
2. *All optimal policies achieve the optimal value function: $v^{\pi^*}(s) = v^*(s)$*
3. *All optimal policies achieve the optimal action-value function: $q^{\pi^*}(s, a) = q^*(s, a)$*

Proposition 3.19 (Finding an Optimal Policy). *An optimal policy can be found by maximizing over $q^*(s, a)$:*

$$\pi^*(s, a) = \begin{cases} 1 & \text{if } a = \arg \max_{a \in \mathcal{A}} q^*(s, a) \\ 0 & \text{otherwise} \end{cases} \quad (16)$$

Corollary 3.20. 1. *There is always a deterministic optimal policy for any MDP*

2. *If we know $q^*(s, a)$, we immediately have the optimal policy*
3. *There can be multiple optimal policies*
4. *If multiple actions maximize $q^*(s, \cdot)$, we can pick any of these (including stochastically)*

4 Bellman Equations

4.1 Recursive Definition of Value Functions

Proposition 4.1 (Recursive Value Function). *The value function can be defined recursively:*

$$v_{\pi}(s) = \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, \pi] \quad (17)$$

$$= \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s, A_t \sim \pi(S_t)] \quad (18)$$

$$= \sum_a \pi(a \mid s) \sum_r \sum_{s'} p(r, s' \mid s, a) (r + \gamma v_{\pi}(s')) \quad (19)$$

Proposition 4.2 (Recursive Action-Value Function). *The action-value function satisfies:*

$$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \quad (20)$$

$$= \mathbb{E}[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \quad (21)$$

$$= \sum_r \sum_{s'} p(r, s' \mid s, a) \left(r + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s', a') \right) \quad (22)$$

4.2 Bellman Expectation Equations

Theorem 4.3 (Bellman Expectation Equations). *Given an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, p, r, \gamma \rangle$, for any policy π , the value functions satisfy:*

$$v_\pi(s) = \sum_a \pi(s, a) \left[r(s, a) + \gamma \sum_{s'} p(s' \mid a, s) v_\pi(s') \right] \quad (23)$$

$$q_\pi(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid a, s) \sum_{a' \in \mathcal{A}} \pi(a' \mid s') q_\pi(s', a') \quad (24)$$

4.3 Bellman Optimality Equations

Theorem 4.4 (Bellman Optimality Equations). *Given an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, p, r, \gamma \rangle$, the optimal value functions satisfy:*

$$v^*(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} p(s' \mid a, s) v^*(s') \right] \quad (25)$$

$$q^*(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid a, s) \max_{a' \in \mathcal{A}} q^*(s', a') \quad (26)$$

Corollary 4.5. *There can be no policy with a higher value than $v^*(s) = \max_\pi v_\pi(s)$ for all s .*

Proposition 4.6 (Greedy Policy on v^* is Optimal). *If $\pi^*(s, a) = 1$ when $a = \arg \max_{a \in \mathcal{A}} q^*(s, a)$, then applying the Bellman Expectation Equation (24):*

$$q_{\pi^*}(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid a, s) \underbrace{\sum_{a' \in \mathcal{A}} \pi^*(a' \mid s') q_{\pi^*}(s', a')}_{\max_{a'} q^*(s', a')} \quad (27)$$

$$= r(s, a) + \gamma \sum_{s'} p(s' \mid a, s) \max_{a' \in \mathcal{A}} q^*(s', a') \quad (28)$$

5 Solving RL Problems Using the Bellman Equations

5.1 Problem Classes in RL

Definition 5.1 (Policy Evaluation / Prediction). Estimating v_π or q_π is called **policy evaluation** or **prediction**.

- Question: Given a policy, what is my expected return under that behavior?
- Example: Given this treatment protocol, what is my expected outcome?

Definition 5.2 (Control). Estimating v^* or q^* is called **control**, as these can be used for policy optimization.

- Question: What is the optimal way of behaving? What is the optimal value function?
- Example: What is the optimal treatment protocol?

Exercise 5.3. Consider an MDP with states $\mathcal{S} = \{S_0, S_1, S_2\}$, actions $\mathcal{A} = \{a_1, a_2\}$:

- Actions have 0.9 probability of success; with probability 0.1 we remain in the same state
- $R_t = 0$ for transitions ending in S_0 ; $R_t = -1$ for all other transitions

Questions (with $\gamma = 0.9$):

1. What is v_π for $\pi(s) = a_1$ (go right) for all s ?
2. What is v_π for the uniformly random policy?
3. Same questions for $\gamma = 0.0$. What do you notice?

5.2 Bellman Equation in Matrix Form

Proposition 5.4 (Matrix Form of Bellman Equation). *The Bellman value equation for a given policy π can be expressed as:*

$$\mathbf{v} = \mathbf{r}^\pi + \gamma \mathbf{P}^\pi \mathbf{v} \quad (29)$$

where:

$$v_i = v(s_i) \quad (30)$$

$$r_i^\pi = \mathbb{E}[R_{t+1} \mid S_t = s_i, A_t \sim \pi(S_t)] \quad (31)$$

$$P_{ij}^\pi = p(s_j \mid s_i) = \sum_a \pi(a \mid s_i) p(s_j \mid s_i, a) \quad (32)$$

Theorem 5.5 (Direct Solution). *The Bellman equation is linear and can be solved directly:*

$$\mathbf{v} = \mathbf{r}^\pi + \gamma \mathbf{P}^\pi \mathbf{v} \quad (33)$$

$$(\mathbf{I} - \gamma \mathbf{P}^\pi) \mathbf{v} = \mathbf{r}^\pi \quad (34)$$

$$\mathbf{v} = (\mathbf{I} - \gamma \mathbf{P}^\pi)^{-1} \mathbf{r}^\pi \quad (35)$$

Remark 5.6. Computational complexity is $O(|\mathcal{S}|^3)$ —only feasible for small problems. Iterative methods exist for larger problems:

- Dynamic programming
- Monte-Carlo evaluation
- Temporal-Difference learning

5.3 Solving the Bellman Optimality Equation

Remark 5.7. The Bellman optimality equation is **non-linear** due to the max operator. We cannot use direct matrix inversion. Iterative solution methods include:

- Using models / dynamic programming: Value iteration, Policy iteration
- Using samples: Monte Carlo, Q-learning, Sarsa

6 Dynamic Programming

Definition 6.1 (Dynamic Programming). Dynamic programming refers to a collection of algorithms that compute optimal policies given a perfect model of the environment as a Markov decision process.

All dynamic programming methods consist of two components:

1. **Policy evaluation:** computing v_π
2. **Policy improvement:** generating a better policy

6.1 Policy Evaluation

We seek to estimate:

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid s, \pi] \quad (36)$$

Algorithm 1 Iterative Policy Evaluation

```

1: Initialize  $v_0$  (e.g., to zero)
2: repeat
3:   for all  $s \in \mathcal{S}$  do
4:      $v_{k+1}(s) \leftarrow \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid s, \pi]$ 
5:   end for
6: until  $v_{k+1}(s) = v_k(s)$  for all  $s$ 
7: return  $v_\pi = v_{k+1}$ 

```

Theorem 6.2 (Convergence of Policy Evaluation). *Under appropriate conditions (e.g., $\gamma < 1$), iterative policy evaluation converges to v_π .*

6.2 Policy Improvement

Proposition 6.3. *Given a value function v_π , we can improve the policy by acting greedily:*

$$\pi_{new}(s) = \arg \max_a q_\pi(s, a) \quad (37)$$

$$= \arg \max_a \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \quad (38)$$

Theorem 6.4 (Policy Improvement Theorem). *Let π and π' be any pair of deterministic policies such that:*

$$q_\pi(s, \pi'(s)) \geq v_\pi(s), \quad \forall s \in \mathcal{S} \quad (39)$$

Then π' is as good as or better than π :

$$v_{\pi'}(s) \geq v_\pi(s), \quad \forall s \in \mathcal{S} \quad (40)$$

6.3 Policy Iteration

Algorithm 2 Policy Iteration

```

1: Initialize policy  $\pi_0$  arbitrarily
2: repeat
3:   Policy Evaluation: Compute  $v_{\pi_k}$ 
4:   Policy Improvement:  $\pi_{k+1}(s) \leftarrow \arg \max_a \mathbb{E}[R_{t+1} + \gamma v_{\pi_k}(S_{t+1}) \mid s, a]$ 
5: until  $\pi_{k+1} = \pi_k$ 
6: return  $\pi^* = \pi_{k+1}$ 

```

Theorem 6.5 (Convergence of Policy Iteration). *Policy iteration converges to the optimal policy π^* and optimal value function v^* .*

Example 6.6 (Jack's Car Rental). • States: Two locations, maximum of 20 cars at each

- Actions: Move up to 5 cars overnight (cost: \$2 each)
- Reward: \$10 for each car rented, $\gamma = 0.9$
- Transitions: Cars returned and requested randomly via Poisson distribution with parameter λ : $P(n) = \frac{\lambda^n}{n!} e^{-\lambda}$
 - Location 1: average requests = 3, average returns = 3
 - Location 2: average requests = 4, average returns = 2

Remark 6.7 (Variations on Policy Iteration). • Does policy evaluation need to converge to v^{π^*} ?

- Could stop when “close” (threshold on change)
- Could stop after k iterations of evaluation
- Extreme: update policy every iteration ($k = 1$) $\Rightarrow$ Value iteration

6.4 Value Iteration

Proposition 6.8. *Taking the Bellman optimality equation and turning it into an update:*

$$\forall s : v_{k+1}(s) \leftarrow \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a] \quad (41)$$

This is equivalent to policy iteration with $k = 1$ step of policy evaluation between greedy improvements.

Algorithm 3 Value Iteration

```

1: Initialize  $v_0$  arbitrarily
2: repeat
3:   for all  $s \in \mathcal{S}$  do
4:      $v_{k+1}(s) \leftarrow \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a]$ 
5:   end for
6: until  $v_{k+1}(s) = v_k(s)$  for all  $s$ 
7: return  $v^* = v_{k+1}$ 

```

Theorem 6.9 (Convergence of Value Iteration). *Value iteration converges to the optimal value function v^* .*

6.5 Summary of Synchronous DP Algorithms

Problem	Bellman Equation	Algorithm
Prediction	Bellman Expectation Equation	Iterative Policy Evaluation
Control	Bellman Expectation Eq. + Greedy	Policy Iteration
Control	Bellman Optimality Equation	Value Iteration

Remark 6.10 (Complexity). • State-value based: $O(|\mathcal{A}||\mathcal{S}|^2)$ per iteration

- Action-value based: $O(|\mathcal{A}|^2|\mathcal{S}|^2)$ per iteration

7 Extensions to Dynamic Programming

7.1 Asynchronous Dynamic Programming

Standard DP uses **synchronous** updates (all states in parallel).

Definition 7.1 (Asynchronous DP). **Asynchronous DP** backs up states individually, in any order:

- Can significantly reduce computation
- Guaranteed to converge if all states continue to be selected

7.1.1 In-Place Dynamic Programming

Proposition 7.2 (Synchronous vs In-Place Updates). *Synchronous value iteration stores two copies:*

$$\text{For all } s \in \mathcal{S}: \quad v_{new}(s) \leftarrow \max_a \mathbb{E}[R_{t+1} + \gamma v_{old}(S_{t+1}) \mid S_t = s] \quad (42)$$

$$v_{old} \leftarrow v_{new} \quad (43)$$

In-place value iteration stores one copy:

$$\text{For all } s \in \mathcal{S}: \quad v(s) \leftarrow \max_a \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s] \quad (44)$$

7.1.2 Prioritised Sweeping

Definition 7.3 (Prioritised Sweeping). Use the magnitude of the Bellman error to guide state selection:

$$\left| \max_a \mathbb{E}[R_{t+1} + \gamma v(S_{t+1}) \mid S_t = s] - v(s) \right| \quad (45)$$

- Backup the state with the largest remaining Bellman error
- Update Bellman error of affected states after each backup
- Requires knowledge of reverse dynamics (predecessor states)
- Implemented efficiently using a priority queue

7.1.3 Real-Time Dynamic Programming

Definition 7.4 (Real-Time DP). Only update states relevant to the agent. For example, if the agent is in state S_t , update that state value or states expected to be visited soon.

7.2 Full-Width vs Sample Backups

Definition 7.5 (Full-Width Backup). Standard DP uses **full-width backups**:

- Every successor state and action is considered
- Uses true model of transitions and reward function
- Effective for medium-sized problems (millions of states)
- Suffers from curse of dimensionality: $n = |\mathcal{S}|$ grows exponentially with state variables

Definition 7.6 (Sample Backup). **Sample backups** use sample rewards and transitions $\langle s, a, r, s' \rangle$ instead of the full model.

Advantages:

- Model-free: no advance knowledge of MDP required
- Breaks curse of dimensionality through sampling
- Cost of backup is constant, independent of $n = |\mathcal{S}|$

8 Summary

Key concepts covered:

1. **Markov Decision Processes**: Formal framework for sequential decision-making
2. **Objectives**: Different notions of return (undiscounted, discounted, average)
3. **Value functions**: Expected returns conditioned on state (and action)
4. **Optimality**: Optimal value functions and optimal policies
5. **Bellman Equations**: Recursive characterization of value functions
6. **Problem classes**: Evaluation (prediction) and control
7. **Policy Evaluation**: Computing v_π
8. **Dynamic Programming**: Policy iteration and value iteration
9. **Extensions**: Asynchronous methods, in-place updates, prioritized sweeping